
Final Project Report MATH 563

Alexandre St-Aubin
Student ID: 261115785

Jonathan Campana
Student ID: 260985613

Edmund Wu
Student ID: 261033501

Elliot Forcier-Poirier
Student ID: 260989602

Abstract

The frustrating problem of *blurry* images has existed since the invention of cameras. Multiple types of blurs occur for different reasons, such as *camera shake blur*, *motion blur*, or *out of focus blur*. These images are a nuisance, and pose challenges in many applications, such as image content recognition, surveillance, astronomy and many more [6]. Hence, reducing blur, or *deblurring* is crucial in solving the ubiquitous problem of blurred images in a society inundated with visual media. In this continually evolving research area, many methods have been proposed, broadly categorized into those addressing the *non-blind* and *blind* deblurring problems. This article will primarily focus on the former, where the blur kernel is known. We will discuss four algorithms designed to tackle this challenge, highlighting their strengths and weaknesses while providing illustrative examples.

1 Introduction

1.1 Notation

Throughout this paper, we will denote the *proximal operator* of f with parameter λ by

$$\mathbf{prox}_{\lambda f}(v) := \arg \min_x \left\{ f(x) + \frac{1}{2\lambda} \|x - v\|_2^2 \right\} \quad (1)$$

The *indicator function* for a convex set C will be defined as

$$\delta_C(x) := \begin{cases} 0, & \text{if } x \in C \\ +\infty, & \text{if } x \notin C \end{cases} \quad (2)$$

We also denote the *Euclidean Projection* on a set C by

$$\mathbf{P}_C(v) := \arg \min_{x \in C} \|x - v\|_2 \quad (3)$$

1.2 Preliminaries

We first note an important relation between the proximal and subdifferential operators.

Proposition 1. Given a function $f \in \Gamma$, we have

$$\mathbf{prox}_{\lambda f} = (I + \lambda \partial f)^{-1}, \quad (4)$$

where $(I + \lambda \partial f)^{-1}$, for $\lambda > 0$, is called the **resolvent** of the operator ∂f . In particular, the proximal operator is the resolvent of the subdifferential operator [5].

Definition 1 (Maximal Monotone operator). A monotone operator is **maximal** if its graph is not properly contained in the graph of any other monotone operator. If A is monotone (resp. maximal monotone) then so are A^{-1} and λA if $\lambda > 0$.

The next proposition, given in [4], will be useful in the construction of the optimizing algorithms.

Proposition 2. If F is maximal monotone, then $(I + \lambda F)^{-1}(\hat{x})$ exists and is unique for all $\hat{x} \in \mathbb{R}^n$. The value $x = (I + \lambda F)^{-1}(\hat{x})$ of the resolvent is the unique solution of the monotone inclusion

$$\hat{x} \in x + \lambda F(x) \quad (5)$$

Proposition 3 (Proximal operator splitting). If f is separable across two variables, so $f(x, y) = \varphi(x) + \psi(y)$, then

$$\text{prox}_f(x, y) = (\text{prox}_\varphi(x), \text{prox}_\psi(y)) \quad (6)$$

Thus, evaluating the proximal operator of a separable function reduces to evaluating the proximal operators for each of the separable parts, which can be done independently [5].

Proposition 4. (Proximal operator of norms) If $f = \|\cdot\|$ is a general norm, then f^* is the indicator function on the unit ball for the dual norm $\|\cdot\|_*$, namely

$$\|\cdot\|^* = \delta_{\|\cdot\|_* \leq 1}$$

where $\|\cdot\|_*$ is given by

$$\|v\|_* = \sup_x \{\langle v, x \rangle \mid \|x\| \leq 1\}$$

In particular, the *Moreau Decomposition Theorem* gives that

$$\text{prox}_{\|\cdot\|}(v) = v - \mathbf{P}_{\{x: \|x\|_* \leq 1\}}(v)$$

1.3 Derivations of the proximal maps

We provide a quick derivation of the proximal operators of the L1 norm, the L2 norm, the iso norm, and the box prox. By the Moreau decomposition, we naturally obtain the proximal operators of the Fenchel conjugates. Let $b \in \mathbb{R}^d$ be fixed.

- Let $g : y \mapsto t\|y - b\|_1$. We have

$$\text{prox}_g(y) := \arg \min_u \left\{ t\|u - b\|_1 + \frac{1}{2}\|y - u\|_2^2 \right\} = \begin{cases} b & y - b \in [-t, t] \\ y - \text{sign}(y - b) \cdot t & \text{otherwise} \end{cases}$$

We consider first-order optimality conditions, with the following cases:

- If $u - b > 0$, then

$$t - y + u = 0 \quad \Leftrightarrow \quad u = y - t$$

- If $u - b < 0$, then

$$-t - y + u = 0 \quad \Leftrightarrow \quad u = y + t$$

- If $u - b = 0$, then $u = b$ under which by optimality conditions we have

$$0 \in t \cdot [-1, 1] - y + b \quad \Leftrightarrow \quad y - b \in [-t, t]$$

- Now consider $g : y \mapsto t\|y - b\|_2^2$. This g is continuously differentiable so we can take its gradient directly and set it to zero. That is, the steps are:

$$\frac{\partial}{\partial u} \left(t\|u - b\|_2^2 + \frac{1}{2}\|y - u\|_2^2 \right) = 2t(u - b) - y + u = u(2t + 1) - y - 2tb$$

Therefore

$$\text{prox}_g(y) = \frac{y + 2tb}{2t + 1}$$

- For $g : (y_1, y_2) \mapsto t\gamma \|(y_1, y_2)\|_{\text{iso}}$ where

$$\|(x, y)\|_{\text{iso}} = \sum_{k=1}^n (x_k^2 + y_k^2)^{1/2}$$

To compute the proximal operator of g , we use Moreau decomposition (7.5.8). The Fenchel conjugate of g is $g^*(y_1, y_2) = t\gamma \max_{1 \leq k \leq n} (y_{1k}^2 + y_{2k}^2)^{1/2}$ where $y_1, y_2 \in \mathbb{R}^n$ (simplified after elementary conjugacy operations). So we have

$$\mathbf{prox}_g(y_1, y_2) = (y_1, y_2) - \mathbf{prox}_{g^*}(y_1, y_2)$$

where

$$\mathbf{prox}_{g^*}(y_1, y_2) = \arg \min_{(u, v)} \left\{ t\gamma \max_{1 \leq k \leq n} (u_k^2 + v_k^2)^{1/2} + \frac{1}{2} \|(y_1, y_2) - (u, v)\|_2^2 \right\}$$

Remark that $(u_k^2 + v_k^2)^{1/2} = \|(u_k, v_k)\|_2$ for each k . The Fenchel conjugate of the L2 norm is itself (HW3). We also have that for any norm $\|\cdot\|$, the Fenchel conjugate $\|\cdot\|^*$ is the indicator of the unit ball in the dual norm $\delta_{\|\cdot\|^* \leq 1}$. The proximal mapping of the indicator of the unit ball is the projection onto the unit ball for points not in the unit ball. Hence

$$\mathbf{prox}_{g^*}(y_1, y_2) = t\gamma \frac{(y_1, y_2)}{\|(y_1, y_2)\|_2}$$

as long as $(y_{1k}^2 + y_{2k}^2)^{1/2} > 0$. If $(y_{1k}^2 + y_{2k}^2)^{1/2} = 0$, then $y_{1k} = y_{2k} = 0$ so $(u, v) = (0, 0)$. So altogether, for $k = 1, \dots, n$

$$\mathbf{prox}_g(y_{1k}, y_{2k}) = \begin{cases} \left(1 - \frac{t\gamma}{(y_{1k}^2 + y_{2k}^2)^{1/2}}\right)(y_{1k}, y_{2k}) & (y_{1k}^2 + y_{2k}^2)^{1/2} > 0 \\ 0 & \text{otherwise} \end{cases}$$

- For the box prox, $f : x \mapsto t\delta_S(x)$ where $S := \{x : x \in [0, 1]\}$, the proximal operator is the projection. That is,

$$\begin{aligned} \mathbf{prox}_f(x) &= \arg \min_u \left\{ t\delta_S(u) + \frac{1}{2} \|x - u\|_2^2 \right\} \\ &= t \cdot \arg \min_u \left\{ \delta_S(u) + \frac{1}{2t} \|x - u\|_2^2 \right\} \\ &= \arg \min_{u \in S} \frac{1}{2} \|x - u\|_2^2 \end{aligned}$$

2 Algorithms

The algorithms we present are used to solve the non-blind image deblurring problem, which is a convex optimization problem of the form

$$\begin{aligned} &\min_{x, y} f(x) + g(y) \\ &\text{subject to } Ax = y, \end{aligned} \tag{7}$$

where $f, g \in \Gamma$ have inexpensive prox-operators. We are given a vectorized, blurred image b , along with the kernel K used to blur it using the model

$$Kx + \eta = b, \tag{8}$$

where x is the original image, and η the added noise. Our goal is to recover the original image x from the blurred and noisy image b . We formulate the problem in two ways, namely, one which uses the L^1 norm, and the other the L^2 norm. First,

$$\min_x \|Kx - b\|_1 + \delta_S(x) + \gamma \|Dx\|_{\text{iso}}, \tag{9}$$

and second,

$$\min_x \|Kx - b\|_2^2 + \delta_S(x) + \gamma \|Dx\|_{\text{iso}}, \quad (10)$$

where $S = \{x \mid 0 \leq x \leq 1\}$, so that δ_S aims to restrict the range of allowed pixels.

The algorithms are based on a splitting scheme to facilitate computation of the resolvents of the maximal monotone operators. As mentioned in a bit more detail in the project prompt, we have from the fixed point iteration scheme

$$x^k = (I + t\mathcal{F})^{-1}(x^{k-1}) \quad k = 1, 2, \dots$$

where $\mathcal{F}$ is a maximal monotone operator. Splitting algorithms decompose the problem into

$$\begin{aligned} x^k &= (I + t\mathcal{A})^{-1}(z^{k-1}) \\ y^k &= (I + t\mathcal{B})^{-1}(2x^k - z^{k-1}) \\ z^k &= z^{k-1} \rho(y^k - x^k) \end{aligned}$$

where $t > 0$ is the step size and $\rho \in (0, 2)$ is a relaxation parameter. Splitting algorithms are used in lieu of a classic fixed point iteration due to expensive computation costs.

2.1 Primal Douglas-Rachford splitting (Spingarn's method)

This first algorithm, which we will refer to by PDRS, was invented in the 1950s as a method to solve the heat equation [3]. Per (9), let

$$\begin{aligned} f(x) &= \delta_S(x) \\ g(x) &= \|Kx - b\|_1 + \gamma \|Dx\|_{\text{iso}} \end{aligned} \quad (11)$$

and consider the minimization problem

$$\min_{x,y} f(x) + g(y) + \delta_0 \left(\begin{bmatrix} A & -I \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} \right) \quad (12)$$

with the optimality condition

$$0 \in \partial f(x) + \partial g(y) + \partial \delta_0 \left(\begin{bmatrix} A & -I \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} \right) =: \mathcal{F}(x, y)$$

As stated in the literature [4], $\mathcal{F}$ is the subdifferential of closed, convex functions, hence is a maximal monotone operator. Thus, by (5), the zeros of $\mathcal{F}$ are the fixed points of the resolvent of $\mathcal{F}$. Therefore, if we can find the fixed points of $\mathcal{F}$, we'll have found the optimal points. Unfortunately, this particular problem is too computationally expensive, so the strategy is to split $\mathcal{F}$ into two maximal monotone operators $\mathcal{A}$ and $\mathcal{B}$ which have inexpensive proximal operators. Finding the fixed points of $\mathcal{A} + \mathcal{B}$ will be equivalent to finding that of $\mathcal{F}$. Define $\mathcal{A}$ and $\mathcal{B}$ as,

$$\begin{aligned} \mathcal{A}(x, y) &= \partial f(x) + \partial g(y) = \partial(\delta_S(x)) + \partial(\|Ky - b\|_1 + \gamma \|Dy\|_{\text{iso}}) \\ \mathcal{B}(x, y) &= \partial \delta_0 \left(\begin{bmatrix} K \\ D \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} \right) \end{aligned} \quad (13)$$

We now focus on finding the resolvents of $\mathcal{A}$ and $\mathcal{B}$,

$$\begin{aligned} J_{\mathcal{A}} &= (I + \lambda \mathcal{A})^{-1} = \text{prox}_{\lambda(f+g)}(x, y) \quad [\text{by (4)}] \\ &= (\text{prox}_{\lambda f}(x), \text{prox}_{\lambda g}(y)) \quad [\text{by Prop. 3}] \end{aligned} \quad (14)$$

where

$$\text{prox}_{\lambda f}(x) = \text{prox}_{\lambda \delta_S}(x) = \arg \min_u \frac{1}{2\lambda} \|x - u\|_2^2 + \delta_S(x) = \arg \min_{u \in S} \frac{1}{2\lambda} \|x - u\|_2^2 = \mathbf{P}_S(x), \quad (15)$$

which is the euclidean projection operator onto S . It can be computed element-wise by

$$(\mathbf{P}_S(x))_i = \begin{cases} 0, & \text{if } x_i < 0 \\ x_i, & \text{if } x_i \in S = \max\{0, \min\{x_i, 1\}\} \\ 1, & \text{if } x_i > 1 \end{cases} \quad (16)$$

Now, we observe that $g(y)$ described in (11) can be written in a different form by letting

$$g(y_1, y_2, y_3) = \|y_1 - b\|_1 + \gamma \|(y_2, y_3)\|_{\text{iso}} \quad (17)$$

subject to

$$y_1 = Ky \quad \text{and} \quad \begin{bmatrix} y_2 \\ y_3 \end{bmatrix} = Dy \iff \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix} = \begin{bmatrix} K \\ D \end{bmatrix} y$$

Then, by Proposition 3, notice that the proximal operator of g may be obtained by splitting its components into

$$\varphi(y_1) = \|y_1 - b\|_1 \quad \text{and} \quad \psi(y_2, y_3) = \gamma \|(y_2, y_3)\|_{\text{iso}}$$

which gives

$$\mathbf{prox}_{\lambda g}(y_1, y_2, y_3) = (\mathbf{prox}_{\lambda \varphi}(y_1), \mathbf{prox}_{\lambda \psi}(y_2, y_3)) \quad (18)$$

We know that the proximal operator of the L^1 norm is the soft-thresholding operator T_λ , so $\mathbf{prox}_{\lambda \varphi}(y_1)$ is given element-wise as follows,

$$(\mathbf{prox}_{\lambda \varphi}(y_1))_i = (\mathbf{prox}_{\lambda \|\cdot\|_1}(y_1 - b))_i = T_\lambda((y_1 - b)_i) := \begin{cases} (y_1 - b)_i - \lambda, & \text{if } (y_1 - b)_i > \lambda \\ (y_1 - b)_i + \lambda, & \text{if } (y_1 - b)_i < -\lambda \\ 0, & \text{otherwise.} \end{cases} \quad (19)$$

As a result of Proposition 4, $\mathbf{prox}_{\lambda \psi}(y_2, y_3)$, is given element-wise by

$$(\mathbf{prox}_{\lambda \psi}(y_2, y_3))_i = \mathbf{prox}_{\gamma \lambda \|\cdot\|_{\text{iso}}}(y_2, y_3)_i = \alpha_i((y_2)_i, (y_3)_i),$$

where,

$$\alpha_i = \begin{cases} 1 - \frac{\gamma \lambda}{\sqrt{(y_2)_i^2 + (y_3)_i^2}}, & \text{if } \sqrt{(y_2)_i^2 + (y_3)_i^2} > \gamma \lambda \\ 0, & \text{otherwise.} \end{cases}$$

As for the resolvent of $\mathcal{B}$, it corresponds to the projection (x, y) of a vector $(\bar{x}, \bar{y})$ onto the subspace $\{(x, y) \mid [K \ D]^T x = y\}$, expressed as

$$J_{\mathcal{B}} = (I + \lambda \mathcal{B})^{-1} = \left(x = \left(I + [K \ D] \begin{bmatrix} K \\ D \end{bmatrix} \right)^{-1} \left(\bar{x} + \begin{bmatrix} K \\ D \end{bmatrix} \bar{y} \right), y = \begin{bmatrix} K \\ D \end{bmatrix} x \right)$$

The algorithm is included in Appendix A.1.

2.2 Primal-Dual Douglas Rachford Splitting

This second method uses both optimality conditions of the primal problem and the dual problem for the "split". The method takes advantage of the inexpensive computation of the prox-operators of f and g , and the fact that our matrix A makes it easy to compute linear equations.

In the algorithm, we consider the dual problem,

$$\text{maximize } -f^*(-A^T z) - g^*(z) \quad (20)$$

with the optimality conditions for both the primal and the dual problem

$$0 \in \begin{bmatrix} 0 & 0 & A^T & I \\ 0 & 0 & -I & 0 \\ -A & I & 0 & 0 \\ -I & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ w \end{bmatrix} + \begin{bmatrix} 0 \\ \partial g(y) \\ 0 \\ \partial f^*(w) \end{bmatrix}$$

In the above, w is the dual variable and z is the dual multiplier for the constraint $y = Ax$, thus the dual problem is subject to

$$A^T z + w = 0$$

We reduce this to a smaller system by using the relationship $\partial g^* = (\partial g)^{-1}$, which follows from the fact that g is closed and convex, and by the properties of the subgradient and conjugate. We can eliminate y by using the above equality that $\partial g^* = (\partial g)^{-1}$, which would leave us with

$$0 \in \begin{bmatrix} 0 & A^T & I \\ -A & 0 & 0 \\ -I & 0 & 0 \end{bmatrix} \begin{bmatrix} x \\ z \\ w \end{bmatrix} + \begin{bmatrix} 0 \\ \partial g^*(z) \\ \partial f^*(w) \end{bmatrix}$$

We can further reduce this by eliminating w , using a similar equality that $\partial f = (\partial f^*)^{-1}$, we get

$$0 \in \begin{bmatrix} 0 & A^T \\ -A & 0 \end{bmatrix} \begin{bmatrix} x \\ z \end{bmatrix} + \begin{bmatrix} \partial f(x) \\ \partial g^*(z) \end{bmatrix} \quad (21)$$

We can now split the monotone operator we have above to a sum of two monotone operators given as

$$\mathcal{A}(x, z) = \begin{bmatrix} \partial f(x) \\ \partial g^*(x) \end{bmatrix} \quad \mathcal{B}(x, z) = \begin{bmatrix} 0 & A^T \\ -A & 0 \end{bmatrix} \begin{bmatrix} x \\ z \end{bmatrix} \quad (22)$$

Similar to the previous algorithm, we now find the resolvents of $\mathcal{A}$ and $\mathcal{B}$.

As seen before with the resolvent of $\mathcal{A}$, the value $(x, y) = (I + At)^{-1}(\hat{x}, \hat{z})$ is given by

$$x = \mathbf{prox}_{If}(\hat{x}) \quad z = \mathbf{prox}_{Ig^*}(\hat{z})$$

For z , we can apply moreau decomposition to get an alternate expression dealing with the conjugate g^* , thus we have,

$$z = \hat{z} - t \mathbf{prox}_{t^{-1}g}(\hat{z}/t)$$

Simplifying the above, we are left with,

$$z = \hat{z} - \mathbf{prox}_{gt}(\hat{z})$$

This will be a similar computation to the prox operator in algorithm 1, but we have right scalar multiplication in this case.

The resolvent for $\mathcal{B}$ is a linear mapping, so considering resolvents for skew-symmetric linear operators

$$\begin{aligned} (I + t\mathcal{B})^{-1} &= \begin{bmatrix} I & tA^T \\ -tA & I \end{bmatrix}^{-1} = \frac{1}{I - ((tA^T)(-tA))} \begin{bmatrix} I & -tA^T \\ tA & I \end{bmatrix} = \frac{1}{I + t^2A^TA} \begin{bmatrix} I & -tA^T \\ tA & I \end{bmatrix} \\ &= \begin{bmatrix} 0 & 0 \\ 0 & I \end{bmatrix} + \frac{1}{I + t^2A^TA} \begin{bmatrix} I \\ tA \end{bmatrix} \begin{bmatrix} I & -tA^T \end{bmatrix} \\ &= \begin{bmatrix} 0 & 0 \\ 0 & I \end{bmatrix} + \begin{bmatrix} I \\ tA \end{bmatrix} (I + t^2A^TA)^{-1} \begin{bmatrix} I & -tA^T \end{bmatrix}^T \end{aligned}$$

Letting (w, v) be the value of the resolvent of $\mathcal{B}$,

$$\begin{aligned} \begin{bmatrix} w \\ v \end{bmatrix} &= \begin{bmatrix} I & tA^T \\ -tA & I \end{bmatrix}^{-1} \begin{bmatrix} \hat{x} \\ \hat{z} \end{bmatrix} = \left(\begin{bmatrix} 0 & 0 \\ 0 & I \end{bmatrix} + \begin{bmatrix} I \\ tA \end{bmatrix} (I + t^2A^TA)^{-1} \begin{bmatrix} I & -tA^T \end{bmatrix}^T \right) \begin{bmatrix} \hat{x} \\ \hat{z} \end{bmatrix} \\ &= \begin{bmatrix} 0 & 0 \\ 0 & I \end{bmatrix} \begin{bmatrix} \hat{x} \\ \hat{z} \end{bmatrix} + \begin{bmatrix} I \\ tA \end{bmatrix} (I + t^2A^TA)^{-1} \begin{bmatrix} I & -tA^T \end{bmatrix}^T \begin{bmatrix} \hat{x} \\ \hat{z} \end{bmatrix} \\ &= \begin{bmatrix} 0 \\ \hat{z} \end{bmatrix} + \begin{bmatrix} I \\ tA \end{bmatrix} (I + t^2A^TA)^{-1} \begin{bmatrix} I & -tA^T \end{bmatrix} \begin{bmatrix} \hat{x} \\ \hat{z} \end{bmatrix} \\ &= \begin{bmatrix} 0 \\ \hat{z} \end{bmatrix} + \begin{bmatrix} I \\ tA \end{bmatrix} (I + t^2A^TA)^{-1} (\hat{x} - tA^T\hat{z}) \\ &= \begin{bmatrix} 0 \\ \hat{z} \end{bmatrix} + \begin{bmatrix} I \\ tA \end{bmatrix} \frac{\hat{x} - tA^T\hat{z}}{(I + t^2A^TA)} \end{aligned}$$

$$= \begin{bmatrix} 0 \\ \hat{z} \end{bmatrix} + \begin{bmatrix} \frac{\hat{x} - tA^T \hat{z}}{(I + t^2 A^T A)} \\ tA \left(\frac{\hat{x} - tA^T \hat{z}}{(I + t^2 A^T A)} \right) \end{bmatrix} = \begin{bmatrix} \frac{\hat{x} - tA^T \hat{z}}{(I + t^2 A^T A)} \\ \hat{z} + tA \left(\frac{\hat{x} - tA^T \hat{z}}{(I + t^2 A^T A)} \right) \end{bmatrix}$$

Thus we have the solution for both w , v ,

$$w = \frac{\hat{x} - tA^T \hat{z}}{(I + t^2 A^T A)} = \frac{\hat{x}}{(I + t^2 A^T A)} - \frac{tA^T \hat{z}}{(I + t^2 A^T A)} \quad (23)$$

$$v = \hat{z} + tA \left(\frac{\hat{x} - tA^T \hat{z}}{(I + t^2 A^T A)} \right) = \hat{z} + \frac{tA\hat{x}}{(I + t^2 A^T A)} - \frac{t^2 AA^T \hat{z}}{(I + t^2 A^T A)} \quad (24)$$

Notice in the algorithm, we will let $\hat{x} = 2x^k - p^{k-1}$ and $\hat{z} = 2z^k - p^{k-1}$. The Primal-Dual Douglas Rachford splitting algorithm is included in Appendix A.1.

This algorithm resembles the first algorithm in section 2.1. They are both splitting algorithm that use the prox-operators of f and g . The algorithms differ in the fact that the Primal-Dual Douglas Rachford splitting algorithm uses both the primal and dual optimality conditions, rather than uniquely using the primal as done in section 2.1. This means that we need to use Moreau decomposition for the conjugate of the prox-operator of g . The t and ρ inputs play a very similar role in the algorithm as it does in the algorithms in section 2.1 and 2.3.

2.3 ADMM (Dual Douglas-Rachford)

The dual problem from above

$$\text{maximize}_y -f^*(-A^T y) - g^*(y) \quad (25)$$

has the equivalent form

$$\text{minimize}_z -f^*(-A^T z) - g^*(z) := h(z) + k(z)$$

and it is said that applying the Douglas-Rachford algorithm to this dual problem results in the alternating direction method of multipliers (ADMM) algorithm. That is, ADMM uses the Douglas-Rachford iteration on the dual problem. Optimality is guaranteed when $0 \in \partial h(z) + \partial k(z)$ in which we have two maximal monotone operators with resolvents $\text{prox}_{th}(z)$ and $\text{prox}_{tk}(z)$.

ADMM, like the previous algorithms, is a splitting algorithm. Unlike the previous two algorithms, it has different optimality conditions, that is it uses solely optimality conditions of the dual problem. Apart from this, it relies, as the other algorithms do, on the Douglas-Rachford iteration, with a different update ordering as mentioned in the project prompt. The proximal operators are fundamentally the same despite different inputs and despite using the inverse of the step size t^{-1} . This is can be interpreted by the reliance of the Lagrangian on functions f and g . Another difference is the use of the step size t to update w^k and z^k , whereas previously somewhat analogous updates used the relaxation parameter ρ . It turns out the ρ is still used, but hidden within the update steps.

A form of the Douglas-Rachford iteration on the dual can be derived to be the following, as given in the project prompt:

$$\begin{aligned} \hat{x}^k &= \arg \min_x \left(f(x) + (z^{k-1} - w^{k-1})^T Ax + \frac{t}{2} \|Ax\|^2 \right) \\ v^k &= z^{k-1} - w^{k-1} + tA\hat{x}^k \\ \hat{y}^k &= \arg \min_y \left(g(y) - (v^k + w^{k-1})^T y + \frac{t}{2} \|y\|^2 \right) \\ z^k &= v^k + w^{k-1} - t\hat{y}^k \\ w^k &= w^{k-1} + \rho v^k + (1 - \rho)z^{k-1} - z^k. \end{aligned}$$

In particular, we are interested in the general problem

$$\begin{aligned} & \min_{u,y} f(u) + g(y) \\ & \text{subject to } \begin{bmatrix} I \\ A \end{bmatrix} x + \begin{bmatrix} -I & 0 \\ 0 & -I \end{bmatrix} \begin{bmatrix} u \\ y \end{bmatrix} = 0 \end{aligned}$$

for which we have an augmented Lagrangian given as

$$L(x, u, y, w, z) = f(u) + g(y) + w^T(x - u) + z^T(Ax - y) + \frac{t}{2} (\|x - u\|^2 + \|Ax - y\|^2)$$

which makes no difference in the above formulation as we alternate minimization over x and minimization over (u, y) , so that they are constants in alternation. The pseudo code can be seen in [Appendix A.1](#).

2.4 Chambolle-Pock

The Chambolle-Pock method considers the generic optimization problem

$$\begin{aligned} & \min_{x,y} f(x) + g(y) \\ & \text{subject to } Ax = y, \end{aligned}$$

by finding the solution of

$$\min_x \max_y \langle Ax, y \rangle + f(x) - g^*(y). \quad (26)$$

It is mentioned that in our case, the solution turns out to be the Lagrangian of our generic optimization problem.

The Chambolle-Pock algorithm stands out as the odd one out among the four algorithms. It does not use a relaxation parameter ρ , but two step sizes t and s which depend on the size of the operators in the problem, as mentioned in the project prompt. Furthermore, this algorithm does not require solving a system of linear equations involving $I + A^T A$. We only need vector-matrix multiplications of A and A^T , hence giving a time complexity that is not necessarily the same as the previous algorithms.

Chambolle-Pock is not based on an "optimality condition" as the previous three, but properly chosen step sizes s and t that will guarantee convergence. In particular, as described in the project prompt, by choosing the step sizes s and t such that $st(\|A\|_2^2) < 1$, we can ensure the following convergence:

$$\frac{\|y^k - \hat{y}\|_2^2}{2s} + \frac{\|x^k - \hat{x}\|_2^2}{2t} \leq C \left(\frac{\|y^0 - \hat{y}\|_2^2}{2s} + \frac{\|x^0 - \hat{x}\|_2^2}{2t} \right) \quad (27)$$

In summary, because the sequence of iterates $\{(x^k, y^k)\}_k$ is bounded, by extracting subsequences, convergence to a fixed point is guaranteed.

Despite these differences, it is said that the Chambolle-Pock algorithm is at its core, a form of preconditioned ADMM. Unpacking this formulation, and with the help of the equations

$$y^k = (I + s\partial g^*)^{-1}(y^{k-1} + sAz^{k-1}) \quad z^k = 2x^k - x^{k-1}$$

due to the addition of

$$\frac{1}{2} \left\langle \left(\frac{1}{s} - tAA^T \right) (y - y^{k-1}), y - y^{k-1} \right\rangle$$

to the augmented Lagrangian at the k -th iteration, we see that the updates of y^k and z^k are exactly two of the steps of the algorithm described above. The algorithm proceeds as below. It utilizes the proximal operators of g^* and f , resembling the primal-dual formulation. The algorithm is included in [appendix A.1](#).

3 Tuning and Results

In this section, we discuss on the hyperparameters and the algorithm results. All the resulting plots and heatmaps in sections 3.1 and 3.2 are from the 'cameraman.jpg' image. It could be said that a more thorough approach would involve testing for all the different parameters on a set of images instead of a single one. To gauge the performance of the algorithms, on top of the obtained image, we use an objective function of the form $x \mapsto \|x - x_0\|_2^2$ for some objective x_0 . This function resembles the mean-squared error. It turns out using as objective the blurred image b suffices for decent results, as we will in fact use the function

$$x \mapsto \|Kx - b\|_2^2$$

as objective function, where K is the blurring matrix and b is the blurred image. It is evident that if the blurring matrix blurs the image in a way that does not resemble the blurred image b at all, this objective function will not serve its purpose, hence the need of choosing a best K . Such a K will naturally depend on the blurred image. So in fact, we do not want to pick a best K , but merely one that gives acceptable results for any blurred image b . This is the function we used in most of our tests. However, we also implemented a mean absolute value function that take the 1-norm instead to compare their performance.

Furthermore, we are also interested in the effects of additive noise η and different sizes of kernels. We will try different choices of additive noise and compare the results. We also investigated the effects of the step size $t > 0$ and the relaxation parameter $\rho \in (0, 2)$. Their effects will be recorded in their subsection. A final parameter of interest is $\gamma > 0$ that appears in the expression $\gamma \|Dx\|_{iso}$. As suggested by the project prompt, this γ will depend on the choice of problem, either l_1 or l_2 . Finally, we also need to consider the difference in performance between l_1 and l_2 .

Every time we initialized the values in our `i` struct which contains all the values for our hyperparameters, we initialized them to 0. We implemented a grid search function to check the different combinations of t and ρ (s, in the case of chambolle-pock). Then, we created a separate file and tested various combinations of parameters to find the best combination, (6^3 total to be precise). We included some heatmaps and plots to demonstrate the performance of various combination of parameters and algorithms in the appendix (A.2 and A.3). The hyper parameters in the table are from the heat maps. The hyperparameter `malength` is explained in section 3.1.

Table 1: Default Parameters Used

Hyperparameter	In-line Name <code>i</code> .	Value
Kernel	<code>kernel</code>	Motion Blur (50, 0)
Noise	η	Salt & Pepper (0.01)
Total Iteration	<code>maxiter</code>	500
gamma l_1	<code>gamma1</code>	0.01
gamma l_2	<code>gamma2</code>	0.01
early stopping param	<code>malength</code>	32

We notice that a few of the parameters are the same for the l_1 problem as for the l_2 problem. However, some are indeed different.

3.1 Hyperparameter tuning

Blurring kernel and noise

We denote the blurring kernel by K and the noise by η . We will consider the following blurring kernels K

- Gaussian blurring
- Motion blurring

Table 2: Default Parameters Used for the l_1 Problem (left) and for the l_2 Problem (right)

Hyperparameter	In-line Name i .	Value	Hyperparameter	In-line Name i .	Value
Step size t	tprimald _r	0.5	Step size t	tprimald _r	0.25
Relaxation ρ	rhoprimald _r	1.5	Relaxation ρ	rhoprimald _r	1.5
Step size t	tprimalduald _r	1.0	Step size t	tprimalduald _r	1.0
Relaxation ρ	rhoprimalduald _r	1.0	Relaxation ρ	rhoprimalduald _r	1.0
Step size t	tadmm	1.0	Step size t	tadmm	0.5
Relaxation ρ	rhoadmm	0.01	Relaxation ρ	rhoadmm	0.1
s	scp	1.0	s	scp	0.75
t	tcp	1.0	t	tcp	0.75

and the following additive noise η

- Poisson noise
- Salt and pepper noise

We tested varying kernel sizes and our different noise types with our fastest algorithm, ADMM with l_1 and similar results to DRP and DRPD. We observed that Gaussian Blur resulted in higher error than motion blur, similarly for Poisson as opposed to Salt and Pepper noise, where the latter had higher final error after deblurring, this is seen in the following table. The reason for Gaussian blur's higher error is likely due to its more diffuse nature than motion blur which is more structured. As for salt and pepper versus Poisson, the higher error of the former is likely for a similar reason as the Poisson noise follows a distribution as opposed to the salt and pepper which is uniformly random.

We implemented two different types of loss functions, mean squared error and mean absolute error (MAE). For motion blur, the MSE tends to be smaller than the MAE, while for gaussian, it is the opposite. In both of the types of errors, we can see that increasing the kernel size increases the error of the final deblurred image.

Table 3: ADMM l_1 error w/ motion blur different noise and kernel sizes

noise / kernel size	5	10	20	50
Poisson	0.103761	0.433995	0.831227	1.981254
Salt and Pepper	5.456321	5.082865	4.744630	4.572537
Poisson (MAE loss)	1.315056	4.478501	5.331412	8.058971
Salt and Pepper (MAE loss)	5.781919	5.755003	6.811920	9.185187

Table 4: ADMM l_1 error w/ Gaussian blur different noise and kernel sizes

noise / kernel size	5	10	20	50
Poisson	2.574415	5.051331	4.998908	4.998903
Salt and Pepper	5.784421	7.060442	7.054892	6.582341
Poisson (MAE loss)	1.524611	4.826774	4.837622	4.837618
Salt and Pepper (MAE loss)	4.267900	5.838472	7.668699	7.984317

Figure 1: Difference between unblurred (right) and salt and pepper Gaussian blurred image (left)



Gamma

For gamma, we were initially running the grid search with regards to gamma and plotting the various combinations of ρ and t , it turned out that the overall best gamma for all the algorithms and the two norms was 0.01. We provided the plotting of various combinations of ρ and t for DRP and gamma in the appendix to show that they all tend to have lowest error when gamma is very low (A.3). The graphs from other algorithms were similar.

Step size and relaxation parameter

As mentioned previously, for the step size and relaxation parameters we implemented a grid search to graph out the different combinations of ρ and t . We also implemented a heat map to better visualize the data, which shows the errors of various combinations of ρ and t (s and t for Chambolle-Pock) and attributes colours to contrast (A.2). All the plots and heatmaps are with regards to the mean squared error.

3.2 Results

We will present the results of the four algorithms with the default parameters. For this, we have chosen to proceed with *motion blur* with a length of 50 and $\sigma = 0$, and a Salt and Pepper additive noise with a density of 0.01. We also consider the l_1 problem as it seems to be more accurate. For the objective function, we are using the mean squared error.

3.2.1 Image results

The results were as follows. The summaries of the algorithms are as in the following table (5). The output images are as in figure (4). The results are comparable, however algorithm 1 and algorithm 3 seem to perform the best. One other comment is that in the original image, Drake is dripped in formal wear that is striped and in a sense sort of resembles noise. This has an impact on the deblurred image, and indeed makes it harder to recover something super clear. In particular, without including MATLAB's function `edgetaper` [2], the obtained image would have had an even worse ringing effect, as seen in figure (3).

We notice that the run time for DRPD is longer than that of DRP and CP. Observing the iterations of the DRPD algorithm, the error would be monotonically decreasing but would do so by a very small amount. Thus it would do the full 500 iterations before coming to a stop. The result, however,

Figure 2: Original image (left) and blurred image (right) [1]



Figure 3: Deblurred (left: no edgetaper, right: edgetaper)



Table 5: Summary of the Algorithms

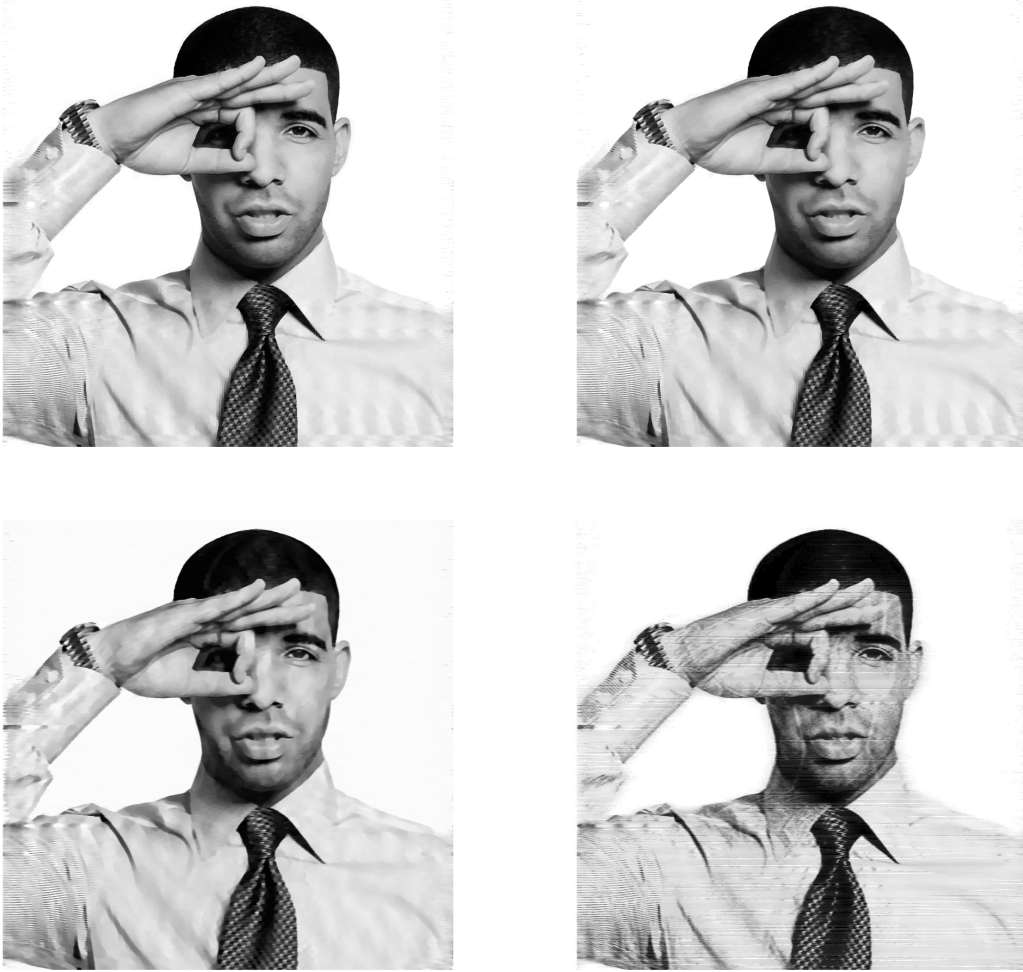
Algorithm	# of Iterations	Run Time	Final Objective
DRP	200	35.09	17.99
DRPD	170	41.25	16.94
ADMM	79	15.85	21.82
CP	34	10.11	179.56

was favourable, where the deblurred image was fairly clear. The ADMM algorithm shared a similar property, but the for loop would end up stopping and we would get the desired result.

The two algorithms with the lowest final objective value were that of DRP and DRPD. This can be seen by figure (5). Furthermore, we observe that the loss updates of Chambolle-Pock is the smoothest,

while DRPD is relatively smooth. The updates for DRP and ADMM are relatively choppy as shown in figure (19). The results of each of the algorithm is as in figure (4).

Figure 4: Results (top left: DRP, top right: DRPD, bottom left: ADMM, bottom right: CP)



3.2.2 l_1 vs l_2 and Early Stopping

We found that l_1 almost always performs better than l_2 (A.2 A.3). Also, l_2 displays much more sensitivity to outliers. This can be explained by the squaring of the terms and the facts that n -norms tend to augment their largest elements disproportionately as they n increases.

Moreover, the l_1 norm is known to induce sparsity. It therefore induces more clearer edges as approximately close values become the same, while the l_2 norm as a penalty does not clips values to 0, or in this case to say b . In some sense, when we implement the proximal operators, the problem " l_1 " or " l_2 " sort of becomes the penalty, and the $\frac{1}{2}\|y - u\|_2^2$ sort of becomes our actual function. In addition, because we are in a non-blind framework, we have knowledge of the blurring kernel K and therefore know of the map say $x \mapsto \|Kx - b\|_1$. Clipping to 0 when $u = b$ as shown in the derivation of the proximal operator of the l_1 norm reminds the sparsity-inducing feature of this map. It naturally leads to clearer edges. As an example, see figure (5).

We are also implementing a form of early stopping based on a moving average. That is, we are keeping a history of the previous `i.malength` number of previous objective value, then taking the sample average, and stopping if the current objective value is greater than that of the sample mean.

Figure 5: Douglas-Rachford Primal with l_1 (left) and l_2 (right)



3.3 Conclusion

We experienced a few problems when completing the project, which first and foremost included learning the programming language. With our group having little to no experience with Matlab, we had to familiarize ourselves with the syntax of the language. With syntax resembling a mix of python and R, and plenty of helpful functions with insightful documentation/instructions, we were quickly able to adjust. We had issues that had us scratching our heads caused by not using *clearvars*. Another task that posed a challenge was parameter tuning. With four algorithms to adjust and 2 norms to choose from in each algorithm, we had to spend sometime optimizing parameters. Implementing heat maps made it easier to find the parameters for the respective algorithms.

The winner of the algorithms would be between the DRP and ADMM algorithms. They both worked quickly and had very nice results. In terms of which algorithm was easier to code, it is tricky to say, as we had very little experience with Matlab when coding the first algorithm, DRP, but we built up experience in time for writing ADMM. However, DRP algorithm is objectively easier to understand and code when comparing it to the implementation of the ADMM algorithm. With that being said, we would recommend the DRP algorithm to an individual with less experience with Matlab and convex optimization, and recommend the ADMM algorithm the seasoned veterans of the subject.

Acknowledgments

Alexandre St-Aubin (Student ID: 261115785) I wrote the initial code to get everyone started, including some proximal and image handling functions, as well as the main function. I wrote code of the Chambolle-Pock algorithm. I wrote the introduction section, apart from the *Derivations of the proximal maps*, which Edmund did. I also wrote the abstract and the entire section on Primal Douglas-Rachford (2.1), along with the introduction to the algorithm section (2). Finally, I was the one who made the heat maps.

Jonathan Campana (Student ID: 260985613) I was in charge of everything related to algorithm 2, the Primal Dual Douglas-Rachford algorithm. Coding the algorithm, the code for computing the conjugate of the prox of a function, and the derivation and explanation of the algorithm in the project report. I also wrote the conclusion on the project report.

Elliot Forcier-Poirier (Student ID: 260989602) I worked on the proof of the Proximal operator of the Iso-Norm and the l_1 norm. I also implemented the grid search for ρ and t (s and t for Chambolle-Pock) and plotted their results, also tested different Kernel sizes, motion vs gaussian blur and poisson vs saltandpepper noise. I wrote part of the results section in the report. I compared the performance between the l_1 and l_2 implementations of the problem and the MAE vs MSE loss functions.

Edmund Wu (Student ID: 261033501) I mainly implemented algorithm 1 and algorithm 3, the initialization of the algorithms, some of the code for the proximal maps, the code to evaluate the objective value, and the plot loss function. I also commented most of the code, wrote the proofs for the proximal operators, wrote the summaries for algorithm 3 and algorithm 4, contributed in the grid search and in the heat map, and contributed in the results section.

References

- [1] Check your neck: A gq dress-shirt primer. URL <https://www.gq.com/gallery/drake-collar-dress-shirt-primer-slideshow>. Accessed: 2024-04-19.
- [2] Avoid ringing in deblurred images. URL <https://www.mathworks.com/help/images/avoid-ringing-in-deblurred-images.html#>. Accessed: 2024-04-19.
- [3] Jim Douglas and H. H. Rachford. On the numerical solution of heat conduction problems in two and three space variables. *Transactions of the American Mathematical Society*, 82(2):421, Jul 1956. doi: <https://doi.org/10.2307/1993056>. URL <https://typeset.io/pdf/on-the-numerical-solution-of-heat-conduction-problems-in-two-41bdiqc5i8.pdf>.
- [4] Daniel O'Connor and Lieven Vandenbergh. Primal-dual decomposition by operator splitting and applications to image deblurring. *SIAM Journal on Imaging Sciences*, 7(3):1724–1754, 2014. doi: [10.1137/13094671X](https://doi.org/10.1137/13094671X). URL <https://doi.org/10.1137/13094671X>.
- [5] Neal Parikh. Proximal algorithms. *Foundations and Trends in Optimization*, 1(3):127–239, 2014. doi: <https://doi.org/10.1561/24000000003>.
- [6] Ruxin Wang and Dacheng Tao. Recent progress in image deblurring, 2014.

Algorithm 4: Chambolle-Pock Algorithm

Input : Blurred image b , step sizes t, s
Output : Deblurred image x
Initialize : x^0, y^0, z^0
1 **for** $k = 1, 2, \dots$, **do**
2 $y^k = \text{prox}_{sg^*}(y^{k-1} + sAz^{k-1})$;
3 $x^k = \text{prox}_{tf}(x^{k-1} - tA^T y^k)$;
4 $z^k = 2x^k - x^{k-1}$;
5 **end**
6 **return** $x\text{Sol} = x^k$;

A.2 Heat Maps

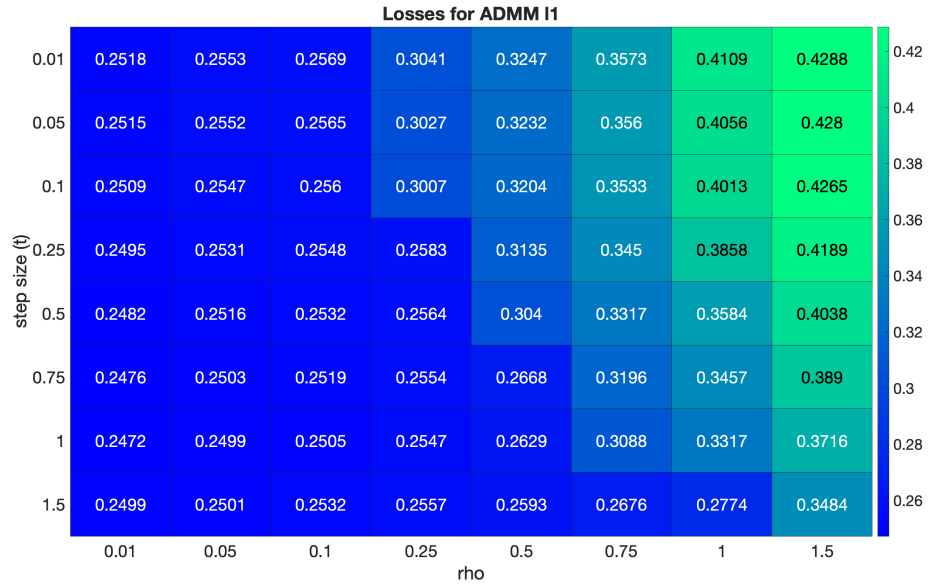


Figure 6: Losses for ADMM with l_1 norm

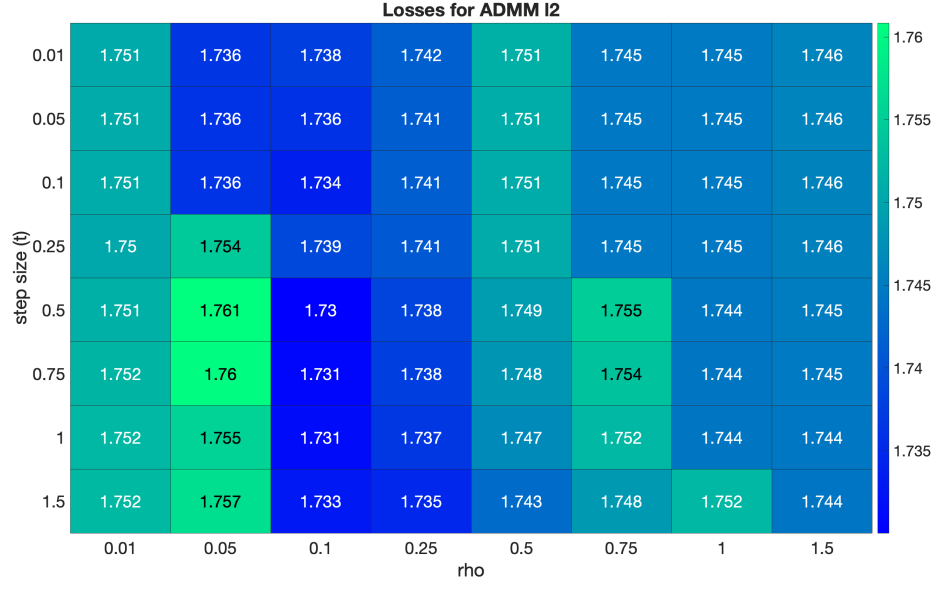


Figure 7: Losses for ADMM with l_2 norm

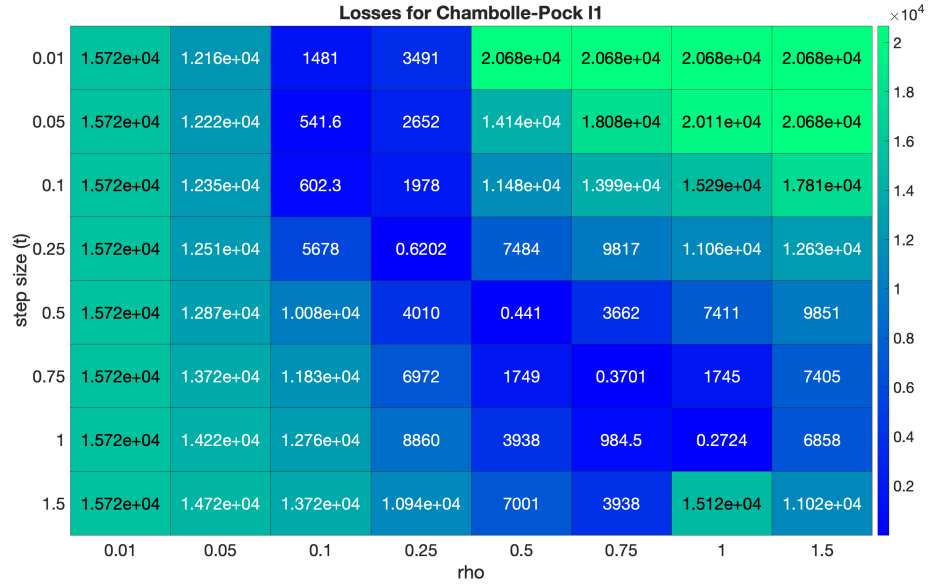


Figure 8: Losses for Chambolle-Pock with l_1 norm (step size = t, rho = s)

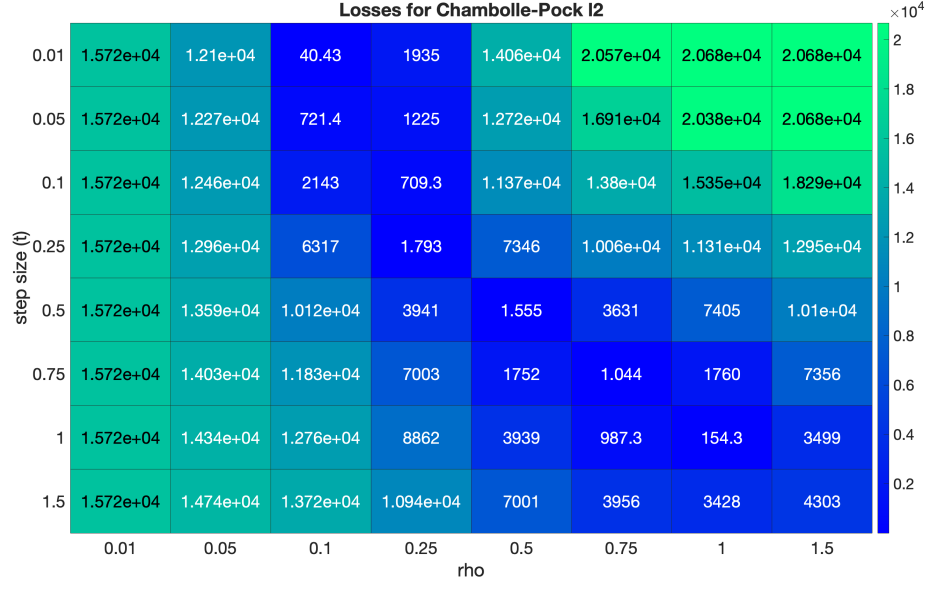


Figure 9: Losses for Chambolle-Pock with l_2 norm (step size = t, rho = s)

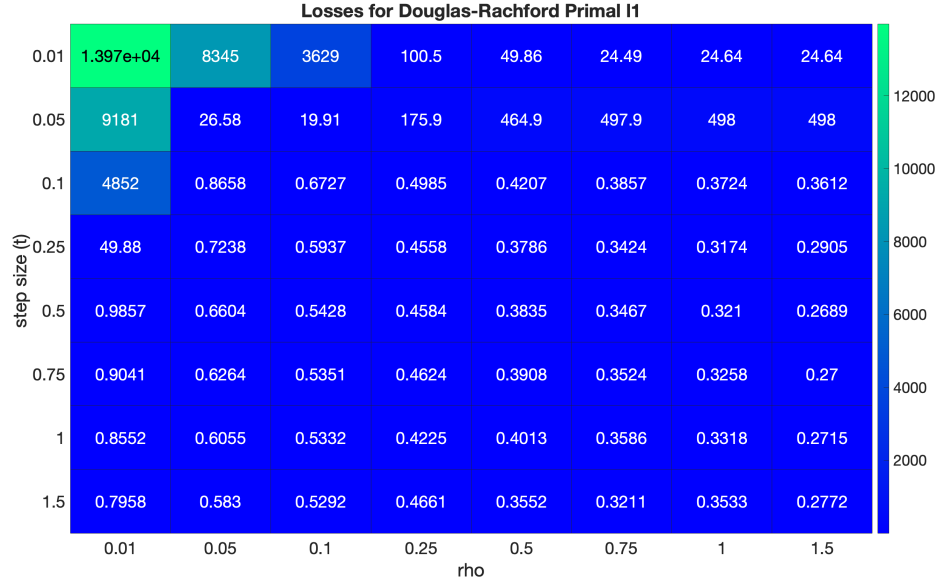


Figure 10: Losses for Douglas-Rachford Primal with l_1 norm



Figure 11: Losses for Douglas-Rachford Primal with l_2 norm



Figure 12: Losses for Douglas-Rachford Primal-Dual with l_1 norm



Figure 13: Losses for Douglas-Rachford Primal-Dual with l_2 norm

A.3 Grid Search Plots

(next page)

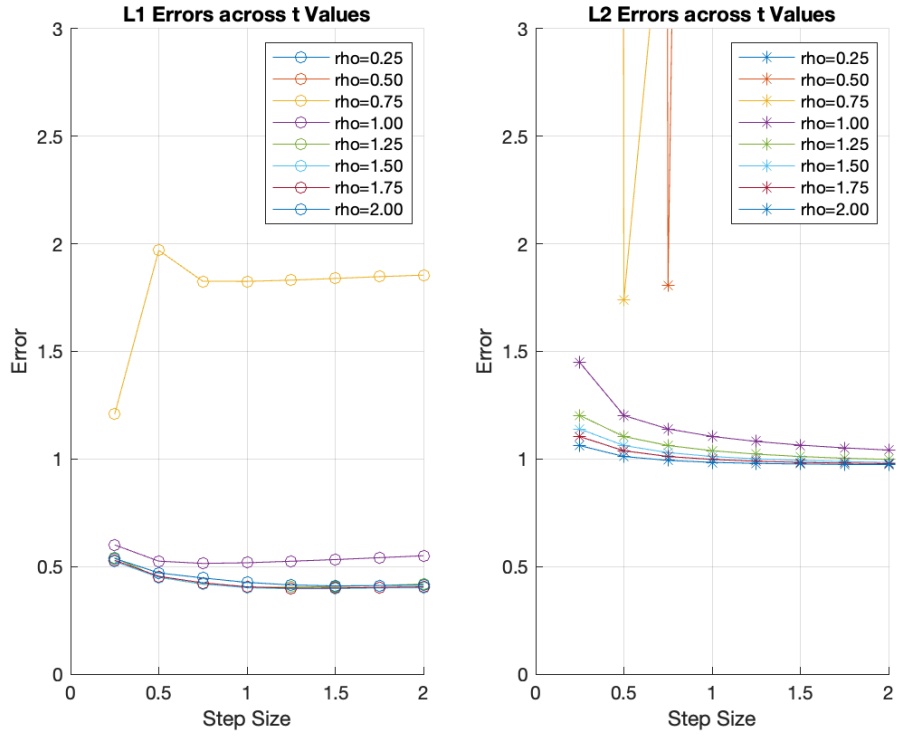


Figure 14: Losses for DRP

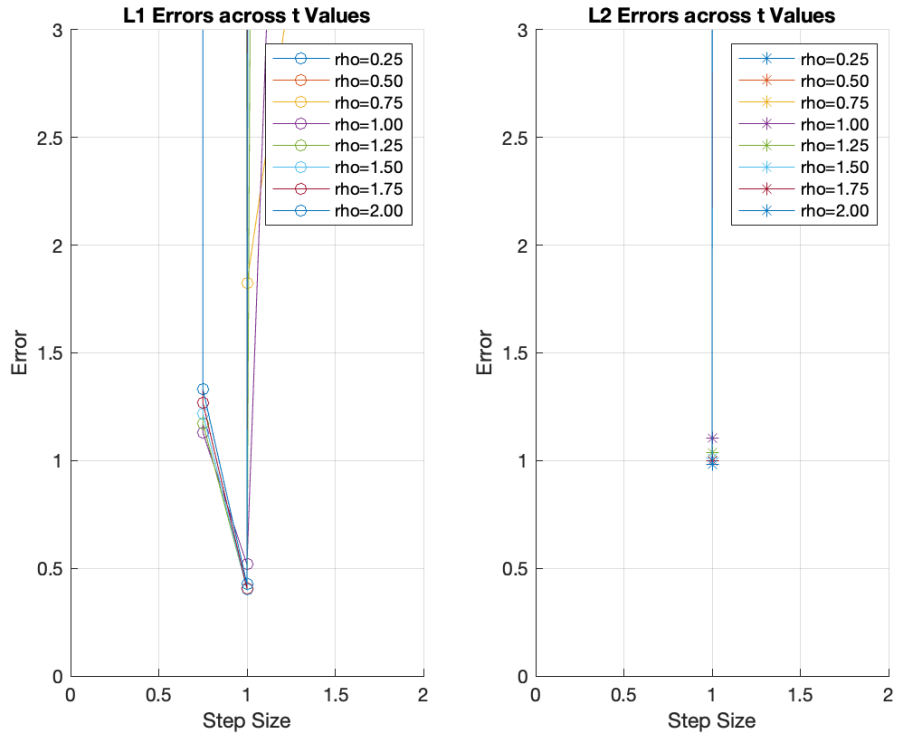


Figure 15: Losses for DRPD (they might be misleading so we refer to the heatmap)

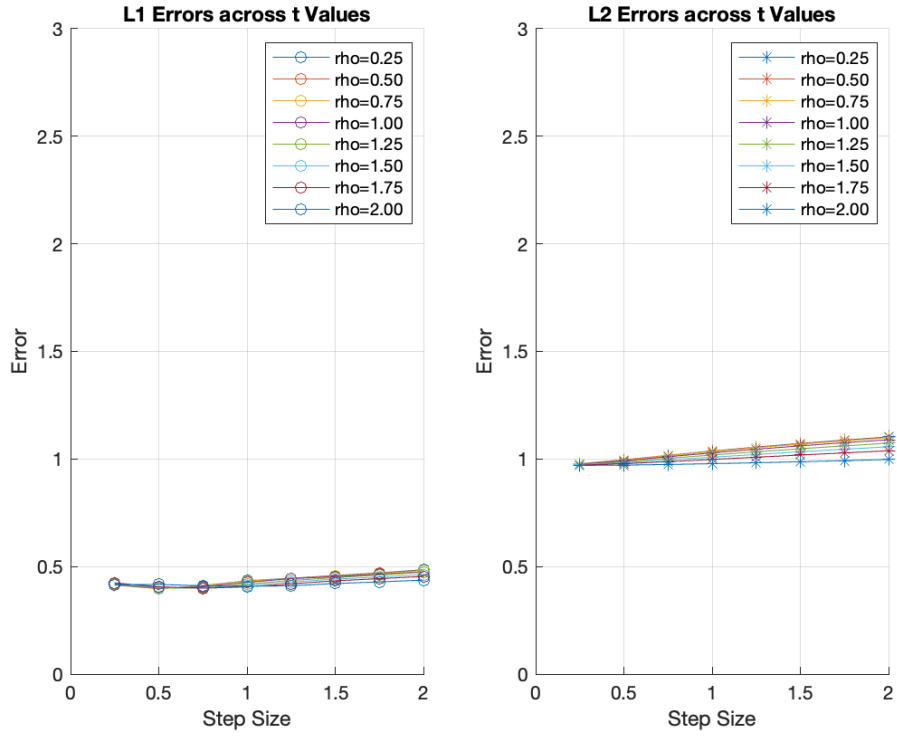


Figure 16: Losses for ADMM

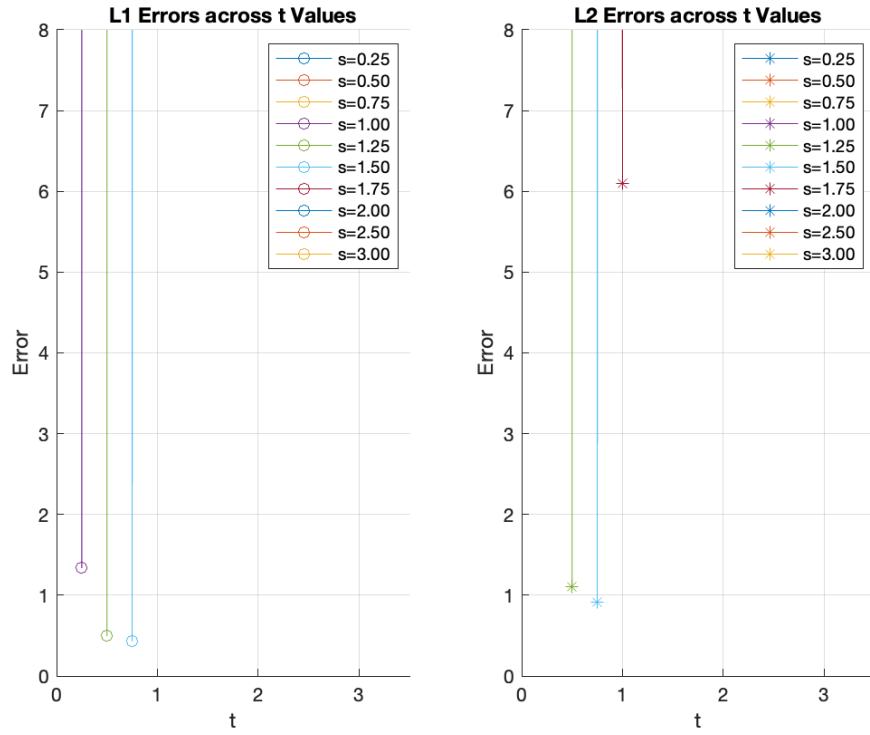


Figure 17: Losses for Chambolle-Pock (they might be misleading so we refer to the heatmap)

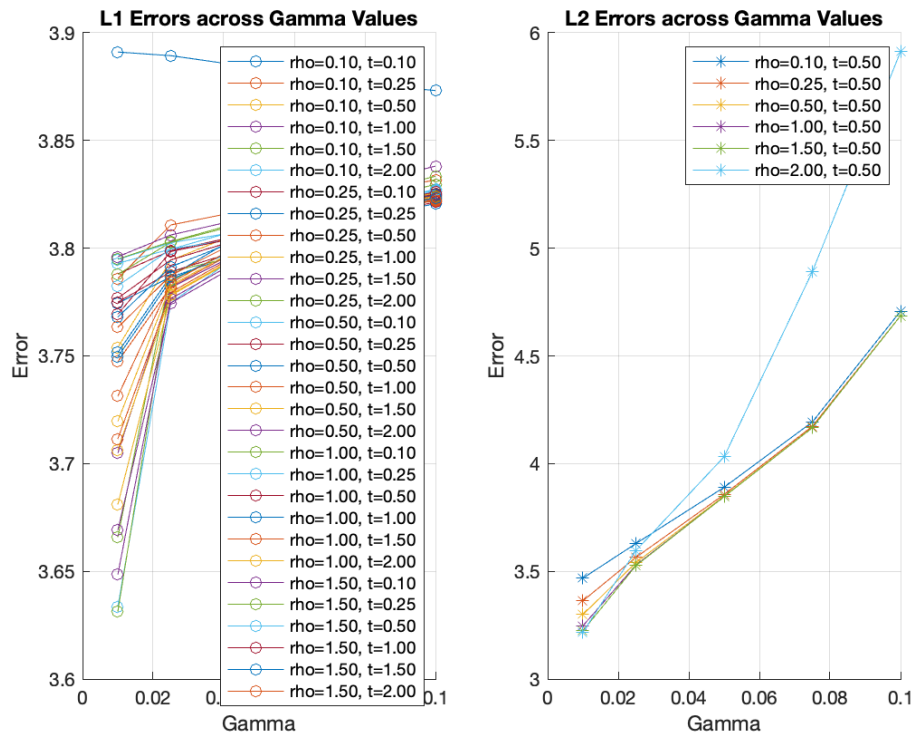


Figure 18: Losses for DRP w/ gamma as x

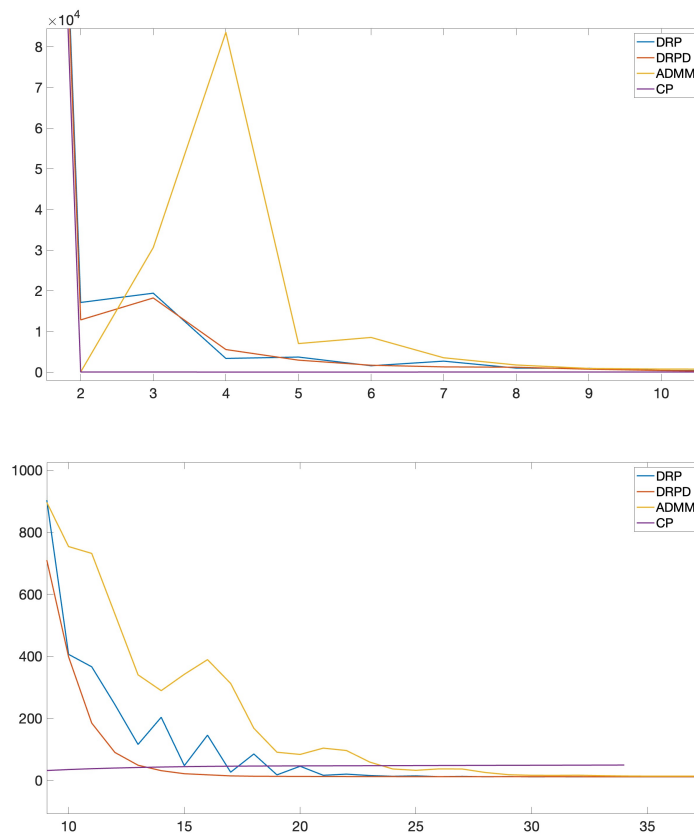


Figure 19: Loss for Drake.png. Note that the above figure plots the loss for a run that does not correspond to the obtained images.